



Instagram

Balint Alex

Berciu Oana-Georgiana

Malan Andreea

Cuprins

- 1.Arhitectura backend
- 2.Tech Stack-backend
- 3.Arhitectura frontend
- 4.Tech Stack-frontend

1.Backend Architecture

Backend-ul aplicației este implementat folosind framework-ul **Spring Boot** și respectă o arhitectură de tip **Layered Architecture (Controller–Service–Repository)**, care separă responsabilitățile în componente distincte. Această organizare contribuie la lizibilitatea codului, testabilitate și mentenanță ușoară.

Structura proiectului este împărțită în mai multe pachete, fiecare având un rol bine definit.

1.1 Pachetul controller

Pachetul controller conține clasele responsabile pentru expunerea endpoint-urilor REST ale aplicației. Acestea reprezintă punctul de intrare pentru cererile HTTP venite din frontend.

Clasele principale sunt:

- PostController
- CommentController
- UserController

Aceste controllere gestionează operații precum:

- crearea, editarea și ștergerea postărilor;
- adăugarea și gestionarea comentariilor;
- autentificarea și gestionarea utilizatorilor.

Controller-ele nu conțin logică de business complexă, ci delegă această responsabilitate către servicii.

1.2 Pachetul service

Pachetul service conține logica de business a aplicației. Aici sunt implementate regulile care guvernează funcționarea sistemului.

Clasele principale sunt:

- PostService
- CommentService
- UserService

- FileStorageService

Responsabilitățile acestui layer includ:

- validarea datelor;
- aplicarea regulilor de acces (ex: doar autorul poate modifica o postare);
- gestionarea relațiilor dintre entități;
- coordonarea operațiilor între controller și repository.

Acest nivel reprezintă „creierul” aplicației.

1.3 Pachetul dao (repository)

Pachetul dao conține interfețele de tip repository, utilizate pentru accesul la baza de date. Acestea sunt implementate automat de Spring prin intermediul **Spring Data JPA**.

Clasele principale sunt:

- UserRepository
- PostRepository
- CommentRepository
- TagRepository

Acestea oferă metode pentru:

- salvarea datelor;
- interogarea datelor;
- ștergerea și actualizarea înregistrărilor.

1.4 Pachetul model

Pachetul model conține entitățile aplicației, care sunt mapate pe tabelele din baza de date.

Entitățile principale sunt:

- User
- Post
- Comment
- Tag

- PostStatus (enum pentru starea postării)

Aceste clase definesc structura datelor și relațiile dintre ele, cum ar fi:

- relația dintre utilizator și postări;
- relația dintre postări și comentarii;
- asocierea postărilor cu tag-uri.

1.5 Pachetul dto

Pachetul dto (Data Transfer Objects) este utilizat pentru transferul datelor între frontend și backend.

Exemple:

- CreatePostRequest
- PostResponse
- UpdateCommentRequest
- UserProfileResponse

Aceste clase au rolul de:

- a evita expunerea directă a entităților din baza de date;
- a controla exact ce date sunt trimise sau primite;
- a structura clar request-urile și response-urile.

1.6 Pachetul security

Pachetul security conține componentele necesare pentru autentificare și autorizare.

Clasele principale:

- JwtAuthenticationFilter
- JwtService

Acestea implementează autentificarea bazată pe **JWT (JSON Web Token)**, asigurând faptul că:

- utilizatorii trebuie să fie autentificați pentru a accesa funcționalitățile;
- fiecare cerere este validată pe baza token-ului.

1.7 Pachetul config

Pachetul config conține clase de configurare pentru aplicație:

- SecurityConfig – configurarea mecanismelor de securitate;

- ApplicationConfig – configurări generale;
- CloudinaryConfig – configurarea serviciului de stocare a imaginilor.

Acest pachet permite definirea comportamentului global al aplicației.

1.8 Pachetul exception

Pachetul exception conține mecanismul de tratare a erorilor la nivel global.

- GlobalExceptionHandler gestionează excepțiile apărute în aplicație și returnează răspunsuri coerente către client.

2.Tech Stack-backend

Pentru dezvoltarea aplicației s-a utilizat un stack tehnologic modern, specific aplicațiilor web de tip client-server. Acesta este format dintr-un frontend dezvoltat în React, un backend implementat în Java folosind Spring Boot și o bază de date relațională PostgreSQL.

Am ales aceste tehnologiile astfel încât să asigure performanță, scalabilitate și o structură clară a aplicației.

2.1 Backend

Backend-ul aplicației este implementat folosind **Java** și framework-ul **Spring Boot**, care facilitează dezvoltarea rapidă a aplicațiilor web și a serviciilor REST.

Spring Boot este utilizat pentru:

- definirea endpoint-urilor REST;
- implementarea logicii de business;
- validarea datelor primite de la client;
- gestionarea autentificării și autorizării;
- integrarea cu baza de date.

De asemenea, sunt utilizate următoarele componente din ecosistemul Spring:

- **Spring Web** – pentru crearea API-urilor REST;
- **Spring Data JPA** – pentru interacțiunea cu baza de date;
- **Spring Security** – pentru securizarea aplicației.

2.2 Baza de date

Pentru stocarea datelor aplicației se utilizează **PostgreSQL**, un sistem de gestiune a bazelor de date relaționale.

PostgreSQL este folosit pentru:

- stocarea utilizatorilor;
- gestionarea postărilor;
- stocarea comentariilor;
- gestionarea tag-urilor și a relațiilor dintre acestea.

2.3 Securitate

Pentru securizarea aplicației se utilizează autentificarea bazată pe JWT (JSON Web Token).

- utilizatorii trebuie să fie autentificați pentru a accesa funcționalitățile;
- fiecare cerere conține un token care este validat pe backend;
- parolele sunt stocate în baza de date în formă criptată (ex: folosind BCrypt).

Această abordare asigură protecția datelor și respectarea cerințelor aplicației.

3. Frontend Architecture

Frontend-ul aplicației este dezvoltat folosind **React** și respectă o arhitectură component-based, specifică aplicațiilor Single Page Application (SPA). Interfața este împărțită în componente reutilizabile, fiecare responsabilă pentru o anumită funcționalitate.

Structura principală a aplicației conține următoarele fișiere:

- **App.jsx** – componenta principală a aplicației, responsabilă pentru routing și organizarea paginilor.
- **main.jsx** – punctul de intrare al aplicației React.
- **Login.jsx** – pagina de autentificare a utilizatorului.
- **Register.jsx** – pagina de înregistrare.
- **Feed.jsx** – pagina principală unde sunt afișate postările utilizatorilor.
- **CreatePost.jsx** – componentă pentru crearea unei postări noi.
- **PostDetails.jsx** – afișarea detaliată a unei postări și comentariilor.
- **Profile.jsx** – pagina de profil a utilizatorului.

- **index.css** – stilizarea globală a aplicației.

Aplicația comunică cu backend-ul prin cereri HTTP către endpoint-urile REST dezvoltate în Spring Boot.

Structura modulară oferă:

- separarea clară a responsabilităților;
- reutilizarea componentelor;
- mentenanță ușoară;
- scalabilitate.

4.Tech Stack-frontend

Frontend-ul aplicației a fost dezvoltat utilizând React, o bibliotecă modernă JavaScript pentru construirea interfețelor dinamice.

Tehnologiile utilizate includ:

- **React** – pentru dezvoltarea componentelor UI;
- **JavaScript (ES6+) & JSX** – pentru logica aplicației;
- **React Hooks** – gestionarea stării componentelor;
- **Tailwind CSS** – stilizare modernă și responsive;
- **Fetch API** – comunicarea cu backend-ul REST;
- **JWT Authentication** – gestionarea sesiunii utilizatorului;
- **React Hot Toast** – notificări interactive;
- **Vite** – tool pentru build și development.

